

Operan ADOS

End-to-End Customer Lifecycle: From First Subscription to Account Closure

How 20 modules work together to replace an enterprise IT offshore department

 Foundation (M01-02)

 Core Runtime (M03-08)

 Governance (M09-11)

 New (M06, M10, M12-19)

 Experience Portal (M21)

 Kafka Event Bus

Table of Contents

Phase 1: Customer Onboarding & Tenant Creation

Phase 2: Identity Setup & Security Configuration

Phase 3: Department Deployment & Template Selection

Phase 4: Knowledge Ingestion & Intelligence Building

Phase 5: Daily Operations & Agent Execution

Phase 6: Monitoring, Optimization & Governance

Phase 7: Account Expiry, Renewal or Closure

Module Interaction Summary Matrix

Kafka Event Topic Reference

1 Customer Onboarding & Tenant Creation

The customer signs a contract. Your sales team creates a tenant. This phase provisions the isolated digital environment, sets quotas, assigns a region (data residency), and creates root admin credentials.



Modules & Their Roles

M21 — Experience Portal

Customer Action: Sales/admin logs into `https://operan.acmecorp.com`

What Happens: Portal proxies to M01's CreateTenant endpoint.

Customer enters company name, plan type (Enterprise), region (me-central for data residency), and contact details.

HTTP Flow: M21 → POST /v1/tenants → M01

M01 — Tenant Control Plane

What Happens: Creates Tenant record with unique namespace, encryption keys (AES-256-GCM), quota limits, and environment policy.

DB Action: INSERT INTO tenants (tenant_id, name, plan_type, region, encryption_key_id, quota)

Output: Returns tenant_id (e.g., "acme-corp-001") and JWT signing secret for this tenant's isolated space.

Kafka Event Bus

Event Published:

`operan.tenant.provisioned`

Payload: { tenant_id, name, plan_type, region, created_by, timestamp }

Consumers: M02 (creates IAM entities), M11 (monitors health), M17 (initializes cost tracking)



Persistent State (PostgreSQL):

```
INSERT INTO tenants VALUES ('acme-corp-001', 'Acme Corp', 'enterprise', 'me-central', 'aes-256-gcm-key-id-...')
```



Kafka Events Published:

`operan.tenant.provisioned` → consumed by M02, M11, M17

`operan.tenant.created` → consumed by M03 (reserved orchestration slots)

The customer configures their identity infrastructure. They enroll users via SSO (Azure AD, Okta), register their first agents, define RBAC policies, and set up MFA. This is the security perimeter setup that gates everything else.



Modules & Their Roles

M21 — Experience Portal

Customer Action: Admin goes to "Identity & Access" tab in portal

What Happens: Portal displays M02's user management UI. Admin configures SSO settings, adds users, assigns roles.

HTTP Flow: M21 → GET /v1/users → M02

M02 — Identity & Access

What Happens: Creates IAM entities: Users, Service Accounts, Agent Identities.

SSO Integration: Connects to Azure AD / Okta via SCIM. Pushes user groups to Operan.

Agent Registration: Admin defines agent identities with roles (e.g., "sales-agent", "contract-reviewer") and capabilities.

RBAC/ABAC: Defines policies: "Sales agents can draft contracts but not send them"

M01 — Tenant Control Plane

Validation: M02 calls M01 to verify tenant has active plan and hasn't exceeded user quota.

Quota Check: GET /v1/tenants/{id}/quota → M01 returns available seats

Kafka Event Bus

Events Published:

`operan.iam.user.created` — M02 publishes new user events

`operan.iam.agent.registered` — M02 publishes agent identity events

`operan.iam.sso.sync` — External SSO directory sync events



Persistent State (PostgreSQL):

```
INSERT INTO users VALUES ('u-001', 'acme-corp-001', 'john@acme.com', 'admin',  
'oidc-azure-ad')  
  
INSERT INTO agent_ids VALUES ('agent-sales-001', 'acme-corp-001', 'sales-  
assistant', ['draft_contracts', 'crm_lookup'])
```



Kafka Events Published:

operan.iam.user.created → consumed by M11, M10
operan.iam.agent.registered → consumed by M04, M11

3

Department Deployment & Template Selection

The customer browses the marketplace or uses a pre-built template to deploy a department. This phase provisions agents, workflows, memory, and governance — the entire department comes alive with one click.



Modules & Their Roles

M21 — Experience Portal

Customer Action: Sales admin goes to "Departments" → "Deploy Template"

What Happens: Portal shows catalog from M15 (Marketplace) with templates: Sales, HR, Finance, Legal.

HTTP Flow: M21 → GET /v1/listings → M15 → returns "Sales Department v2.1"

M05 — Department Template Engine

What Happens: Loads Sales Department template which defines:

- 3 agents: sales-assistant, contract-reviewer, pricing-analyst
- 2 workflows: opportunity-to-contract, quote-generation
- Governance rules, KPI definitions, memory topology

Deployment Pipeline: select → configure → connect_data → provision_memory → deploy_swarm →

M04 — Agent Registry

What Happens: Template defines 3 agents. M05 calls M04's POST /v1/agents to register each:

- sales-assistant: capabilities=[draft_contracts, crm_lookup]
- contract-reviewer: capabilities=[review_contracts, risk_assessment]
- pricing-analyst: capabilities=[quote_generation, margin_analysis]

M03 — Agent Orchestration

What Happens: M05 calls M03's POST /v1/workflows to create DAGs:

- opportunity-to-contract: fetch_crm → draft_contract → pricing_check → human_gate → send_contract
- quote-generation: fetch_products → calculate_price → generate_quote → email_quote

State: Workflows created in "pending" state, awaiting memory and policy setup.

M10 — Policy Governance

What Happens: Template defines governance rules. M05 calls M10's POST /v1/policies:

- Rule 1: deny send_email if cost > \$250k (requires M09 approval)
- Rule 2: allow draft_contracts for sales-agent role
- Rule 3: proxy (human review) for all contract changes > 10% margin

M17 — Cost Governance

What Happens: M05 creates budget based on template's cost estimates:

- Monthly budget: \$5,000 for sales department
- Soft limit (80%): warn at \$4,000
- Hard limit (95%): throttle at \$4,750

M09 — Human Supervision

What Happens: M05 configures human gates from template:

- Gate 1: contract-sending requires manager approval (threshold: 1 approver)
- Gate 2: changes > 10% margin require director approval (threshold: 2 approvers)

M12 — Model Abstraction

What Happens: Template specifies which models to use:

- Contract drafting: gpt-4o (high quality)
- Quote generation: claude-3-haiku (cost-optimized)

M05 calls M12's POST /v1/models to register these model mappings.

M14 — Agent Collaboration

What Happens: M05 creates channels for the department:

- Channel: "sales-dept-001" (type: department)
- Members: all 3 agents + human supervisor
- Agents can now send messages, hand off tasks, share context

Kafka Event Bus

Events Published:

operan.templates.template_deployed → M15, M11
operan.agent-registry.agent_registered → M04 confirms, M11 tracks
operan.orchestration.workflow_created → M03 confirms, M11 tracks
operan.supervision.gate_configured → M09 confirms gates
operan.cost.budget_created → M17 confirms budget



Persistent State (PostgreSQL):

```
INSERT INTO agents (name, role, capabilities, tenant_id) VALUES ('sales-assistant', 'sales', [...], 'acme-corp-001')
```

```
INSERT INTO workflows (name, steps, tenant_id) VALUES ('opportunity-to-contract', [...], 'acme-corp-001')
```

```
INSERT INTO policies (name, action, scope, resource_type) VALUES ('no-large-emails', 'deny', 'tenant', 'tool')
```

```
INSERT INTO budgets (budget_amount, period, soft_limit_pct, hard_limit_pct) VALUES (5000, 'monthly', 80, 95)
```



Kafka Events Published:

operan.templates.template_deployed → consumed by M15, M11
operan.agent-registry.agent_registered → consumed by M04, M11, M14
operan.orchestration.workflow_created → consumed by M03, M11, M14

The customer uploads their documents: CRM exports, past contracts, compliance manuals, pricing guides, customer preference documents. M06 ingests, chunks, normalizes (Arabic if needed), embeds, and stores everything in M07's memory fabric for semantic search.



Modules & Their Roles

M21 — Experience Portal

Customer Action: Admin goes to "Knowledge" → "Upload Documents"

What Happens: Portal shows file upload UI, SharePoint connector config, URL ingestion form.

HTTP Flow: M21 → POST /v1/sources → M06

M06 — Knowledge Ingestion

What Happens: Creates ingestion source (file, URL, SharePoint, email). Starts async pipeline:

1. **Extract:** PDF/DOCX/XLSX → text layer
2. **Normalize:** Call M19 for Arabic text (tashkeel removal, alef standardization)
3. **Chunk:** Split text into 512-char chunks with 50-char overlap
4. **Dedup:** SHA-256 hash to skip duplicate content
5. **Embed:** Call M12 for vector embeddings
6. **Store:** Call M07 to store vectors + metadata

Progress: WebSockets show real-time: "Processed 47/120 chunks..."

M19 — Arabic Language Core

Called When: Source has `normalize_arabic=true`

What Happens: Normalizes Arabic text: removes tashkeel, standardizes alefs/hamzas, detects dialect (MSA vs Gulf vs Egyptian).

Endpoint: POST /v1/normalize → {

M12 — Model Abstraction

Called When: After chunking, before storing

What Happens: M06 calls M12's POST /v1/models/embeddings for each chunk.

Response: { embedding: [768 floats], model: "qwen3-embedding-4b", tokens: 120 }

```
normalized_text, actions: [...], is_msa:
true }
```

M07 — Memory Fabric

Called When: After embedding, to persist vectors

What Happens: M06 calls M07's vector store API to store chunks with metadata:

```
INSERT INTO memory_vectors
(tenant_id, namespace, vector,
metadata) VALUES ('acme-corp-001',
'sales-dept', [...], {source:
'contract-2024.pdf', page: 3, chunk:
47})
```

Search Ready: Agents can now query M07 for "which interface language does the client prefer?" — semantic match returns right result even with zero word overlap.

M19 — Arabic Terminology Governance

Optional: M06 also calls M19's terminology check:

```
POST /v1/terminology/check → {
  flagged_terms: [{term:
    "deprecated_term", suggestion:
    "approved_term"}] }
```

Purpose: Ensures ingested documents use approved government terminology.

Kafka Event Bus

Events Published:

operan.knowledge.ingestion_started → M11 tracks

operan.knowledge.chunk_created → M11 tracks

operan.knowledge.embedding_stored → M07 acknowledges

operan.knowledge.ingestion_completed → M21 updates UI



Persistent State:

```
INSERT INTO ingestion_sources (tenant_id, name, source_type, file_type)
VALUES ('acme-corp-001', '2024 Contracts', 'file', 'pdf')
INSERT INTO ingestion_results (job_id, chunk_index, chunk_text,
```



```
chunk_metadata) VALUES ('job-001', 47, 'Customer prefers Arabic-first UI...',
{source: 'contract.pdf', page: 3})
M07 stores: vector[768] + metadata{tenant: acme-corp-001, dept: sales-dept,
source: contract.pdf, page: 3}
```



Kafka Events Published:

operan.knowledge.ingestion_started → consumed by M11

operan.knowledge.chunk_created → consumed by M11

operan.knowledge.embedding_stored → consumed by M07, M11

5

Daily Operations & Agent Execution

This is the heart of the system. Agents execute workflows 24/7. Each execution touches multiple modules in a complex dance of policy checks, model calls, tool execution, memory lookups, and human gates. Here's the complete flow for a single real-world scenario: **processing a \$250k contract opportunity**.



Scenario: Processing a \$250K Contract

Step 1: Opportunity Created (M03)

CRM webhook triggers workflow. M03 receives POST

/v1/workflows/{id}/execute

Request: { opportunity_id: "opp-2024-047", value_usd: 250000, customer: "GovCorp" }

Flow: M03 creates DAG execution, starts node 1: fetch_crm_data

State: workflow_id=wf-2024-047, status="executing"

Step 2: Policy Check (M10)

Before executing, M03 calls M10's POST /v1/policies/evaluate:

Request: { agent_id: "sales-assistant", action: "fetch_crm_data", resource: "crm_opportunities", cost: 0 }

M10 Response: { allowed: true, action: "allow", policy_name: "crm-access-sales" }

Audit: M10 logs to policy_audits table, publishes operan.policy.evaluation

Step 3: Tool Execution (M08)

M03 calls M08's POST
/v1/tools/execute with tool call:

Request: { tool_name: "crm_lookup",
params: {id: "opp-2024-047"} }

M08 Actions:

- Calls M10 for policy check (tool execution policy)
- Calls M16 for sandbox isolation (if tool needs execution)
- Executes via M18 connector (Salesforce API)
- Records result, publishes
operan.tool-execution.tool_executed

Step 4: Memory Lookup (M07)

Agent needs customer context. Calls
M07's POST /v1/vectors/search:

Query: { vector: [embedding of
"customer preferences"], top_k: 5,
tenant_id: "acme-corp-001" }

Response: { results: [{text: "Customer
prefers Arabic-first UI...", similarity:
0.92}, ...] }

M07 publishes:

operan.memory.search_completed

Step 5: Model Call (M12 via M13)

Agent needs to draft contract. Calls
M13 for model selection, then M12 for
inference:

M13 Resolution: { model_id: "gpt-4o",
reason: "high quality needed for legal"
}

M12 Inference: POST
/v1/models/completions → returns
drafted contract text

M12 tracks: cost_usd=0.02,
prompt_tokens=500,
completion_tokens=800

M12 publishes:

operan.model.model_call_completed →
consumed by M17

Step 6: Cost Check (M17)

M12's cost event triggers M17 to
update budget:

M17 Action: Updates cost_events
table, re-evaluates budgets

Result: { spent: \$4,520 / \$5,000,
percentage: 90.4%, throttle_level:
"soft" }

Warning: M17 publishes

operan.cost.budget_warning
(approaching 95% hard limit)

Step 7: Human Gate (M09)

Step 8: Human Decision (M21 → M09)

Contract value \$250k exceeds threshold. M03 calls M09's POST /v1/gates/raise:

M09 Action: Creates approval request, publishes
operan.supervision.gate_raised

Notification: M21 shows "New approval request: \$250k contract to GovCorp"

State: workflow pauses at gate, status="waiting_approval"

Manager logs into M21, reviews contract, clicks "Approve":

M21 → M09: POST /v1/gates/respond { gate_id, action: "approve", comment: "Looks good" }

M09 Action: Records approval, publishes
operan.supervision.gate_responded

Flow Resumes: M03 consumes gate_responded event, continues workflow

Step 9: Contract Sent (M03 → M08 → M18)

Workflow resumes. Final node: send_contract via M08 → M18:

M08: Calls M18's SMTP connector to send email with contract attachment

M18 SMTP: Sends email via corporate SMTP relay, tracks delivery status

M08 publishes: operan.tool-execution.tool_executed with success result

Step 10: Agent Coordination (M14)

Contract sent. sales-assistant notifies contract-reviewer via M14:

M14 Action: Posts message to "sales-dept-001" channel, creates handoff: { to_agent: "contract-reviewer", title: "Review sent contract", context: {contract_id: "con-047"} }

M14 publishes:
operan.collaboration.handoff_created → consumed by M03

Step 11: Full Audit (M11)

Throughout execution, M11 consumes all Kafka events and records:

M11 stores: metrics, spans, traces for the entire workflow

M21 displays: Real-time dashboard showing workflow progress, latency, costs

M11 publishes:
operan.observability.span_created for

Kafka Event Bus — All Events

Published during this execution:

operan.orchestration.workflow_created
operan.policy.evaluation
operan.tool-execution.tool_executed
operan.model.model_call_completed
operan.cost.budget_warning
operan.supervision.gate_raised
operan.supervision.gate_responded
operan.collaboration.handoff_created
operan.observability.span_created
operan.orchestration.workflow_completed

every step

operan.orchestration.workflow_completed



Persistent State (PostgreSQL) — M03:

```
INSERT INTO workflows (id, tenant_id, status, nodes) VALUES ('wf-2024-047',  
'acme-corp-001', 'completed', [...])  
INSERT INTO workflow_executions (id, workflow_id, status, started_at,  
completed_at) VALUES ('exec-001', 'wf-2024-047', 'completed', '2024-01-15  
10:00:00', '2024-01-15 10:15:00')
```



Kafka Events — Complete Flow:

```
operan.orchestration.workflow_created → M03, M11  
operan.policy.evaluation → M11  
operan.tool-execution.tool_executed → M11, M17  
operan.model.model_call_completed → M17  
operan.cost.budget_warning → M17 (soft throttle), M11  
operan.supervision.gate_raised → M09, M21  
operan.supervision.gate_responded → M03 (resumes), M11  
operan.collaboration.handoff_created → M03, M11, M14  
operan.observability.span_created → M11  
operan.orchestration.workflow_completed → M11, M17, M09
```

6

Monitoring, Optimization & Governance

After deployment, ongoing operations require monitoring, cost optimization, policy tuning, model performance tracking, and periodic knowledge updates. This phase runs continuously.



Continuous Operations & Modules

M11 — Observability (Always Running)

M13 — Multi-Model Routing (Optimization)

What Happens: Consumes all Kafka events, records metrics, creates distributed traces.

M21 Dashboard: Real-time views of:

- Active workflows: 47 running, 12 completed, 3 paused at gates
- Agent performance: sales-assistant (98% success, avg latency 1.2s)
- Cost tracking: \$4,520 / \$5,000 monthly budget (90%)
- Health: All 21 modules healthy, Kafka lag < 100ms

What Happens: M13 tracks model performance via events, re-ranks models.

Analysis: "gpt-4o has 95% quality but costs \$0.02/token. claude-3-haiku has 92% quality at \$0.002/token."

Action: M13 updates routing rules to use haiku for non-critical tasks (quotations, summaries), reserving gpt-4o for contracts.

Result: 60% cost reduction with <3% quality delta.

M17 — Cost Governance (Continuous)

What Happens: M17 monitors cost_events, enforces budgets.

Alert: "Budget at 95% — hard throttle enabled for new contract workflows"

Action: M17 publishes `operan.cost.throttle_triggered` → M03 pauses non-critical workflows

Resolution: Admin increases budget or approves override via M21.

M10 — Policy Governance (Tuning)

What Happens: Admin reviews policy audit log in M21.

Finding: "30% of contract drafts flagged for human review — threshold too low"

Action: Update policy via M21 → M10: `PATCH /v1/policies/{id} → { action: "proxy", threshold: 15% }`

Effect: Only 10% of contracts now require human review (down from 30%).

M06 — Knowledge Updates (Periodic)

What Happens: Monthly or as new documents arrive.

Process: Admin uploads new pricing guide → M06 ingests → M19 normalizes → M12 embeds → M07 stores.

Effect: Agents immediately have

M15 — Marketplace (Agent Upgrades)

What Happens: Vendor releases updated agent template.

Customer Action: M21 shows "New version available: Sales Agent v3.0 (improved contract accuracy)"

Deploy: One-click upgrade → M15 deploys to M04/M03, version history

access to latest pricing without retraining.

preserved.

M18 — Connector Health (Continuous)

What Happens: M18 monitors connector health, runs periodic syncs.

Alert: "Salesforce connector: 503 error on lookup_account"

Action: M18 retries, notifies admin via M21. If persistent, M10 triggers alert policy.

M21 — Admin Dashboard (Continuous)

What Happens: Central hub for all operations.

Views Available:

- Overview: Tenant stats, department health, KPIs
- Agent Catalog: Browse, activate/deactivate agents
- Workflows: Monitor, pause, resume, replay
- Supervision: Approval queue, escalation management
- Observability: Live metrics, traces, health status
- Cost Dashboard: Real-time budget tracking, alerts
- Policies: Policy rules, audit log, compliance reports



Persistent State Updates (Ongoing):

M11: INSERT INTO spans (trace_id, span_id, service, operation, duration_ms) VALUES (...)

M17: INSERT INTO cost_events (tenant_id, cost_usd, model_name) VALUES ('acme-corp-001', 0.02, 'gpt-4o')

M10: INSERT INTO policy_audits (tenant_id, result, evaluation_ms) VALUES ('acme-corp-001', 'allowed', 12)



Kafka Events (Continuous):

operan.observability.metric_recorded → M11

operan.observability.span_created → M11

operan.cost.cost_recorded → M17

operan.policy.evaluation → M11

operan.connector.sync_completed → M11

operan.model.route.resolved → M13 (performance tracking)

7 Account Expiry, Renewal or Closure

After months or years of operation, the customer's subscription ends. They can renew, downgrade, or close the account. This phase handles data retention, graceful degradation, and compliance requirements (GDPR, data residency).

Modules & Their Roles

M21 — Experience Portal

Customer Action: Admin sees "Subscription Expiring" banner on dashboard.

Options Shown:

- Renew: Same plan, same price
- Upgrade: More agents, more storage
- Downgrade: Fewer agents, reduced budget
- Cancel: Export data, close account

HTTP Flow: M21 → POST
/v1/tenants/{id}/renew → M01

M01 — Tenant Control Plane

Renewal:

- Updates tenant.plan_type, tenant.quota
- Extends tenant.expires_at
- Activates all suspended agents/workflows

Cancellation:

- Sets tenant.status = "suspended"
- Disables all JWT tokens for tenant
- Begins 30-day grace period

Data Retention: Per compliance requirements, retains data for 90 days before permanent deletion.

M17 — Cost Governance

Renewal: Updates budgets, removes hard throttle

Cancellation: Sets budgets to zero,

M09 — Human Supervision

Action: Closes all open approval gates, publishes

operan.supervision.gate_closed

publishes `operan.cost.budget_expired`

Final Report: Generates cost summary for accounting: "Total spend: \$47,832 over 18 months"

Archives: All historical approvals/escalations for audit trail.

M03 — Agent Orchestration

Cancellation:

- Sets all workflows to "cancelled"
- Stops all in-progress executions
- Publishes

`operan.orchestration.workflow_cancelled`

Data: Workflow history preserved in M03 for audit (90 days).

M04 — Agent Registry

Cancellation:

- Sets all agents to `status="inactive"`
- Preserves agent definitions for 90 days
- Can restore if customer re-subscribes

M07 — Memory Fabric

Cancellation (90 days):

- Vectors marked as "expired"
- After 90 days: `DELETE FROM`

`memory_vectors WHERE`
`tenant_id='acme-corp-001'`

Export: Customer can export all memory vectors + metadata before deletion.

M10 — Policy Governance

Cancellation:

- All policies set to `status="deprecated"`
- Audit log preserved for 7 years (compliance requirement)

GDPR: "Right to be forgotten" — customer can request full data deletion.

M11 — Observability

Cancellation:

- Metrics retained for 90 days
- Traces archived for compliance
- After retention period: data purged per retention policy

Kafka Event Bus

Final Events:

`operan.tenant.suspended` → M02, M03, M04, M07, M09, M10, M11, M17

`operan.tenant.data_exported` → M07, M10, M11

`operan.tenant.deprovisioned` → All modules, data deleted



Final State Transitions:

M01: UPDATE tenants SET status='suspended', expires_at='2024-06-15' WHERE tenant_id='acme-corp-001'

M04: UPDATE agents SET status='inactive' WHERE tenant_id='acme-corp-001'

M03: UPDATE workflows SET status='cancelled' WHERE tenant_id='acme-corp-001'

M07: UPDATE memory_vectors SET status='expired' WHERE tenant_id='acme-corp-001'

M10: UPDATE policies SET status='deprecated' WHERE tenant_id='acme-corp-001'



Kafka Events — Closure Flow:

operan.tenant.suspended → consumed by all modules

operan.tenant.data_exported → consumed by M07, M10, M11

operan.tenant.deprovisioned → consumed by all modules, triggers data deletion



Renewal Flow (Simpler Path)

Renewal Steps

1. M21 shows renewal prompt
2. Customer clicks "Renew"
3. M01 extends tenant.expires_at
4. M17 resets cost tracking for new period
5. M09 clears any suspended gates
6. M03 resumes workflows from "paused"
7. M04 reactivates all agents
8. Kafka event: operan.tenant.renewed
9. All modules receive event, resume normal operation



Module Interaction Summary Matrix

Every module's interactions across the entire customer lifecycle. Shows which modules call which, when, and for what purpose.

Module	Phase 1	Phase 2	Phase 3	Phase 4	Phase 5	Phase 6
M01 Tenant	✔ Create tenant	Quota check	Template quota			
M02 IAM	Create IAM	✔ Users, agents	RBAC setup			
M03 Orchestration			✔ Create workflows		✔ Execute DAGs	Monitor, pause
M04 Registry			✔ Register agents		Resolve agents	Agent metrics
M05 Templates			✔ Deploy dept			
M06 Ingestion				✔ Ingest docs	Update knowledge	Periodic ingest
M07 Memory			Provision memory	✔ Store vectors	✔ Semantic search	Memory health
M08 Tools			Register tools		✔ Execute tools	Tool metri
M09 Supervision			✔ Setup gates		✔ Human approval	Supervision logs
M10 Policy	Default policies	RBAC policies	✔ Department rules		✔ Pre-execution checks	Policy tuning
M11 Observability	Tenant events	IAM events	Deploy events	Ingestion events	✔ All execution events	✔ Metric traces

M12 Models			Register models	Embed texts	LLM inference	Cost tracking
M13 Routing			Setup routing		Model selection	Re-rank models
M14 Collaboration			Create channels		Agent messaging	Presence tracking
M15 Marketplace			Browse templates		Deploy agents	Agent upgrades
M16 Sandbox					Isolate execution	Sandbox metrics
M17 Cost	Init tracking		Set budgets		Track costs	Cost optimization
M18 Connectors			Configure		External calls	Connector health
M19 Arabic				Normalize text	Terminology check	Glossary updates
M21 Portal	UI for onboarding	UI for IAM	UI for deploy	UI for ingest	All operations UI	All monitoring



Kafka Event Topic Reference

All Kafka events published by the Operan platform. Each topic follows the pattern `operan.{module}.{action}`. Events flow through Kafka and are consumed by relevant modules.

Tenant & IAM Events

Topic	Module	Trigger	Consumers
operan.tenant.provisioned	M01	Tenant created	M02, M11, M17
operan.tenant.renewed	M01	Subscription renewed	M02-M21 (all)
operan.tenant.suspended	M01	Subscription expired	M02-M21 (all)
operan.iam.user.created	M02	User enrolled via SSO	M11, M10
operan.iam.agent.registered	M02	Agent identity created	M04, M11, M14

Core Runtime Events

Topic	Module	Trigger	Consumers
operan.templates.template_deployed	M05	Department deployed	M15, M11
operan.agent-registry.agent_registered	M04	Agent registered	M11, M14
operan.orchestration.workflow_created	M03	Workflow DAG created	M11, M14
operan.orchestration.workflow_completed	M03	DAG execution finished	M11, M17, M09
operan.tool-execution.tool_executed	M08	Tool execution complete	M11, M17
operan.knowledge.chunk_created	M06	Document chunked	M11
operan.knowledge.embedding_stored	M06	Vector stored in M07	M07, M11
operan.memory.search_completed	M07	Semantic search done	M11

Governance Events

Topic	Module	Trigger	Consumers
operan.supervision.gate_raised	M09	Human approval needed	M09, M21

operan.supervision.gate_responded	M09	Human approved/rejected	M03 (resumes), M11
operan.policy.evaluation	M10	Policy check performed	M11
operan.policy.violation	M10	Policy rule violated	M11, M09 (escalate)
operan.cost.cost_recorded	M12/M08	Model/tool call cost	M17
operan.cost.budget_warning	M17	Budget approaching limit	M17, M11, M03
operan.cost.throttle_triggered	M17	Hard limit reached	M03 (pauses), M11
operan.observability.span_created	M11	Execution step traced	M11 (internal)

New Modules Events

Topic	Module	Trigger	Consumers
operan.model.model_call_completed	M12	LLM inference done	M17, M11
operan.model.route.resolved	M13	Model selected by router	M13 (perf tracking), M11
operan.arabic.text_normalized	M19	Arabic text processed	M11
operan.arabic.terminology_check	M19	Terminology governance	M11
operan.collaboration.message_sent	M14	Agent message posted	M11, M07
operan.collaboration.handoff_created	M14	Task handed off	M03, M11
operan.marketplace.listing_created	M15	New template/agent listed	M11

operan.marketplace.deployed	M15	Agent deployed to tenant	M03, M04, M11
operan.sandbox.execution_completed	M16	Sandbox execution done	M08, M11
operan.connector.sync_completed	M18	Connector sync done	M11



Key Insight: Every single action an agent takes produces a Kafka event. M11 (Observability) consumes ALL events and records them. This creates a complete audit trail and enables real-time monitoring, distributed tracing, and analytics. The event bus is the nervous system that connects all 20 modules.